Playwright - Automation framework

MENU:

I. ***Introduction***

II. ***Feature***

III. ***How To Run***

IV. ***Installation***

a. *Step1: Python Install*

b. *Step2: Install IDE VS Code*

c. *Step3: Install Playwright and Libraries*

V. ***Structure***

VI. ***Explaining***

VII. ***Usage***

a. *config*

b. *API*

c. *Data*

d. *Interfaces*

e. *Actions*

f. *TestSuites*

VIII. ***Dictionary***

a. *utils.py*

b. *page_utils.py*

c. *api_utils.py*

d. *db_utils.py*

❖ Introduction

This Test Automation Framework is created using Python + Playwright + Pytest which helps the tester in automating test execution of web applications - API and database.

❖ Feature

To fit with our issues when write test script to automate, this framework will support main features as below:

- **Web application :** Although playwright framework supports cross-browser, we run only on Chromium at the current stage.
- **API :** Playwright can be used to get access to the REST API, these cookies will be used from the corresponding chromedriver. This means that if we log in using this API, our chromedriver will be logged in and vice versa.
- **Database :** with pyodbc extension, we can automate database testing on local.

❖ How To Run

- Run all case:

  Run this command on Terminal:

  pytest

- Run specific Test Class:

  Run this command on Terminal:

  pytest -k <TestSuitesFileName>

  Ex:

  pytest -k TestSuites_03_LRF_Reports_Win_Loss_Analysis.py

- Run Single Test of Test Class

  Run this command on Terminal:

  pytest -v TestCase/<TestClassname>.py::<TestClassName>::<TestCaseName>

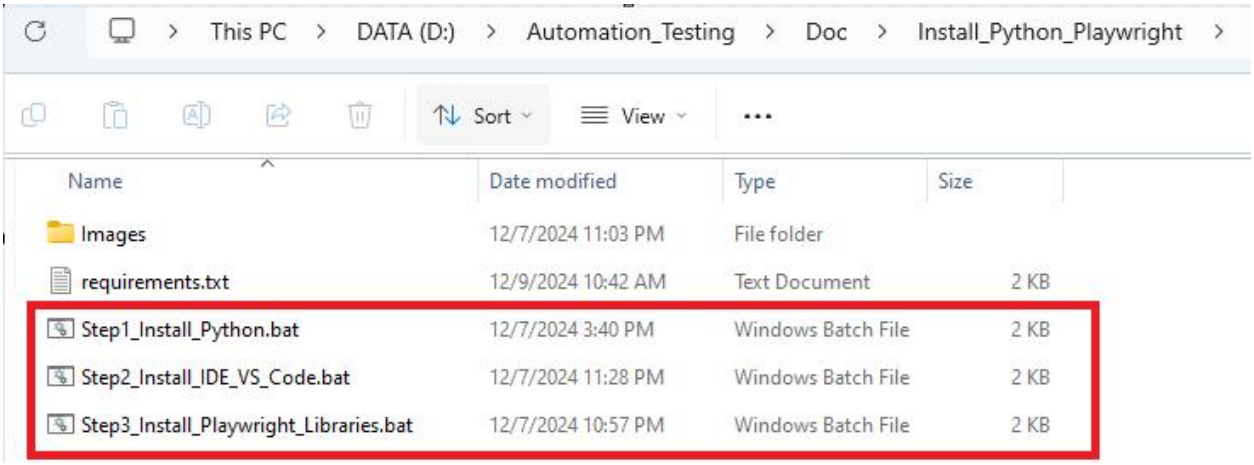
- For Example we would like to run testcase **TestCase_001_Login** on **Commission_Group** of **TestCase_01_Commission_Group.py**:

pytest -v TestCase/TestCase_01_Commission_Group.py::Commission_Group::TestCase_001_Login

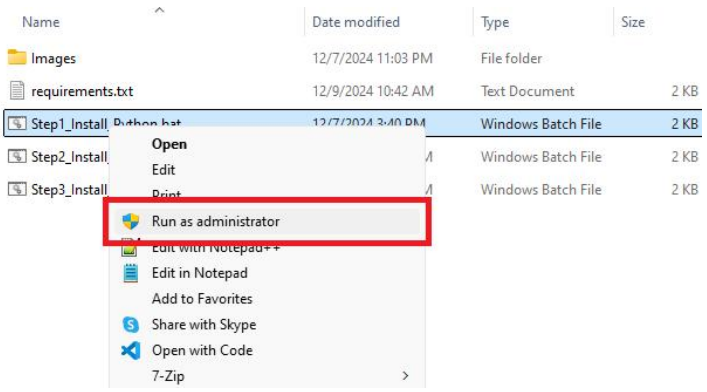
❖ Installation:

➢ Step1: Python Install

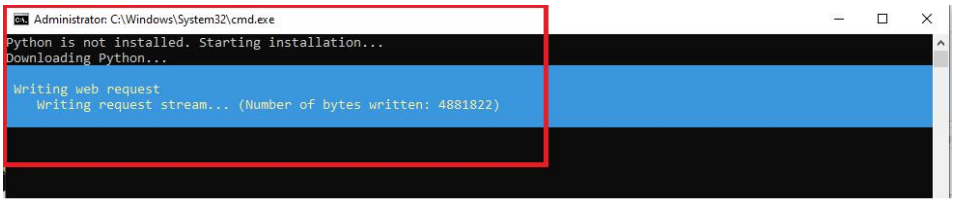
Find the **Install_Python_Playwright** folder in the available source code, and you will see three **.bat** files named according to the steps. Run each file in order of the steps.



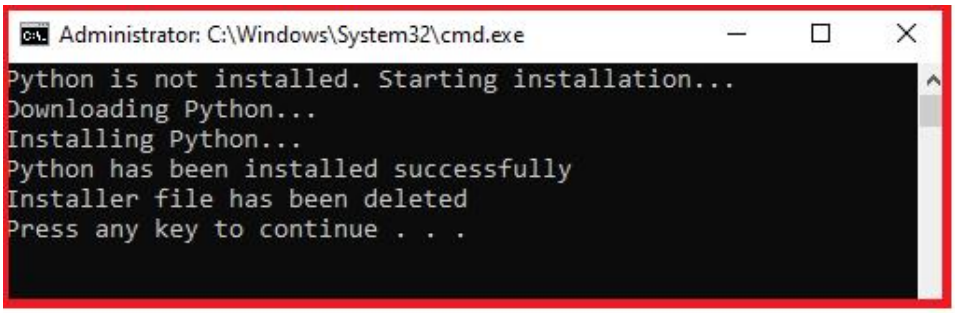
❖ Run Step1_Install_Python.bat with administrator



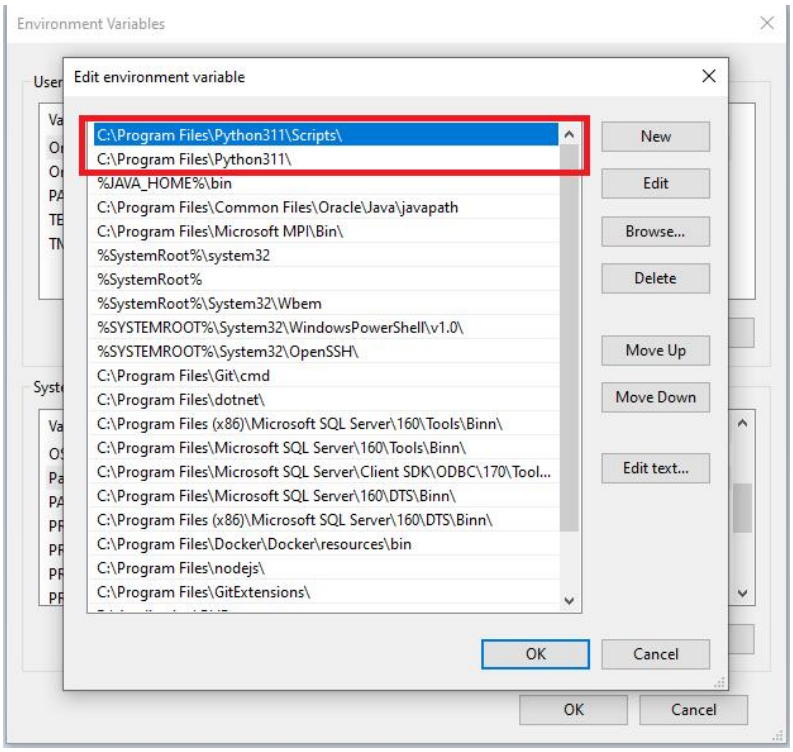
----------RUNNING INSTALL PYTHON----------



If you see a message like the image below, it means Python has been installed and the environment path has been set up successfully.

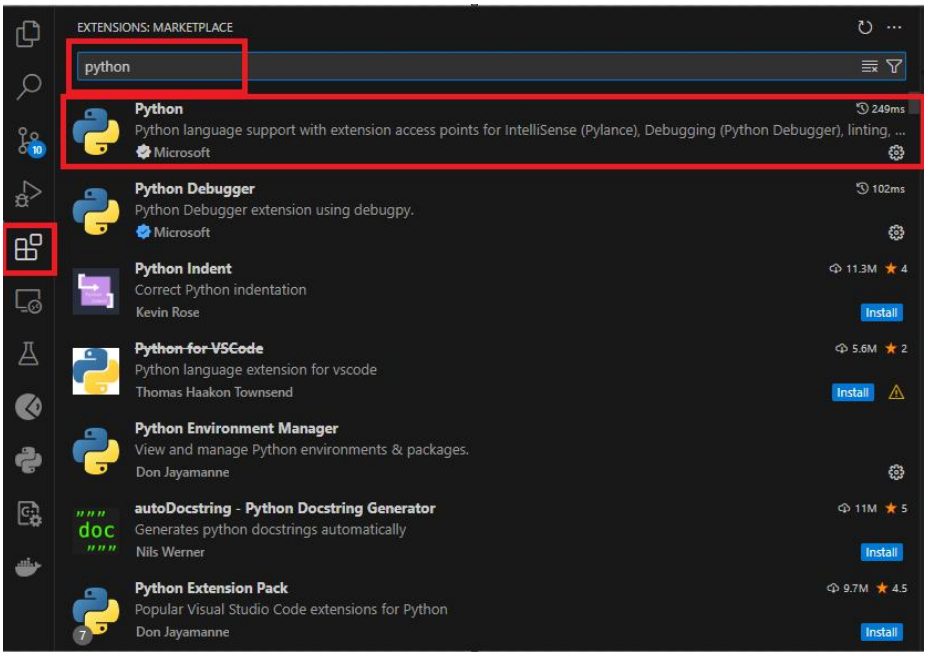


You can go to **Edit the system environment variables** to check if Python has been added to the **Path** in the System Variables. If it matches the image below, the Python installation is complete.



➢ Step2: Install IDE VS Code

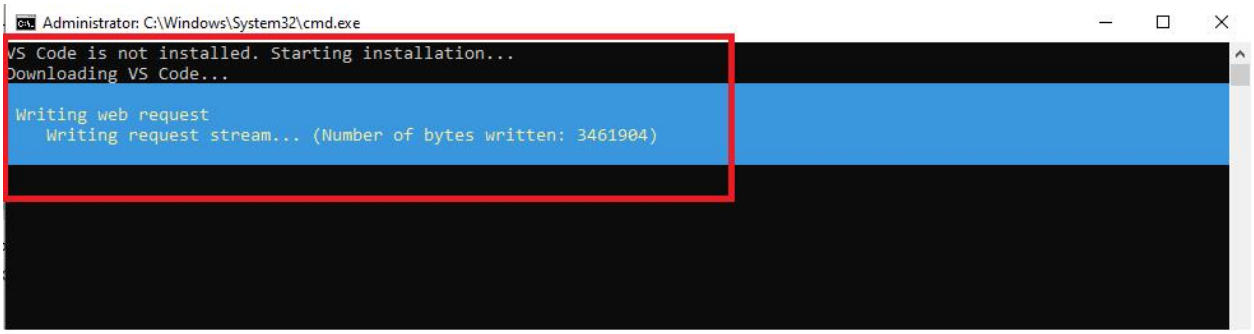
I. If you have installed VS Code, proceed to install the **Python Extension for Visual Studio Code** as shown in the image below.

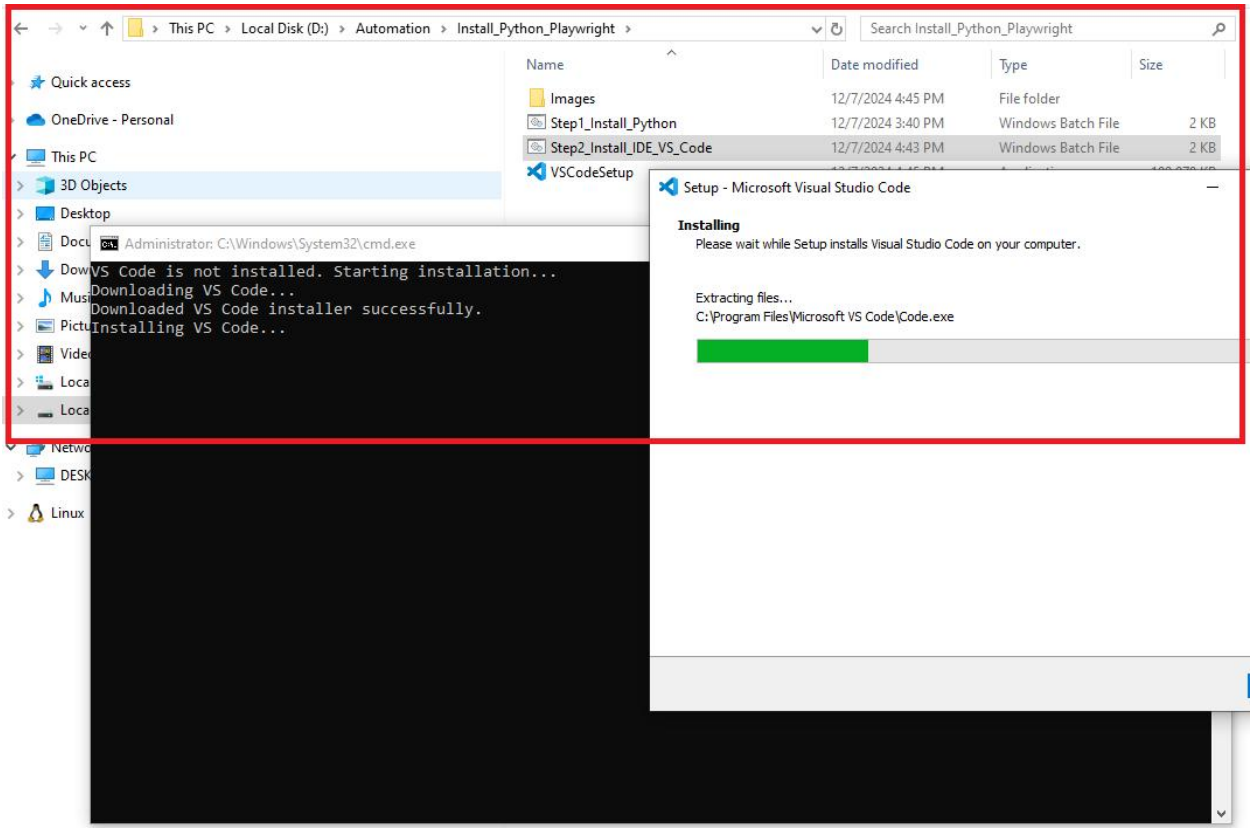


II. If you haven't installed VS Code yet, follow these steps:

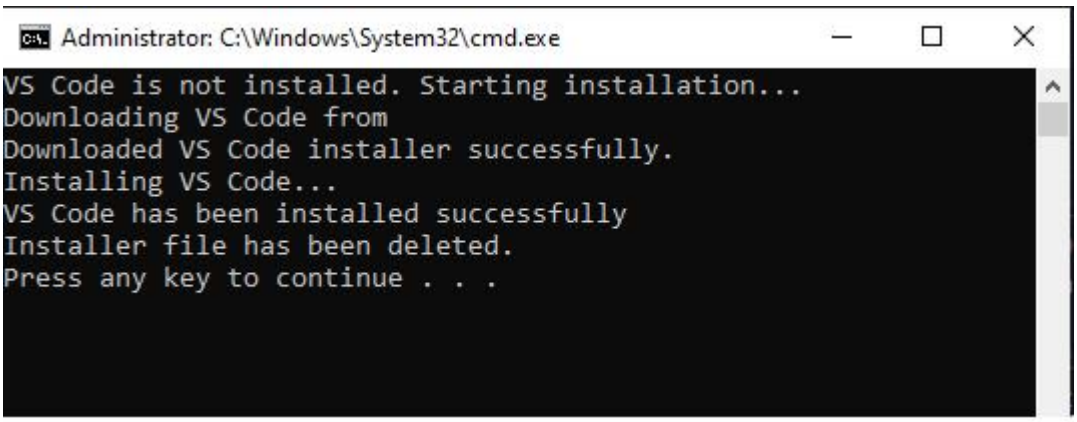
- Run **Step2_Install_IDE_VS_Code.bat with administrator**

---------- RUNNING INSTALL IDE VS CODE----------





If you see a message like the image below, the IDE installation process is complete. Once the installation is successful, VS Code will open automatically. Then, repeat **Step I: Install the Python Extension for Visual Studio Code**.

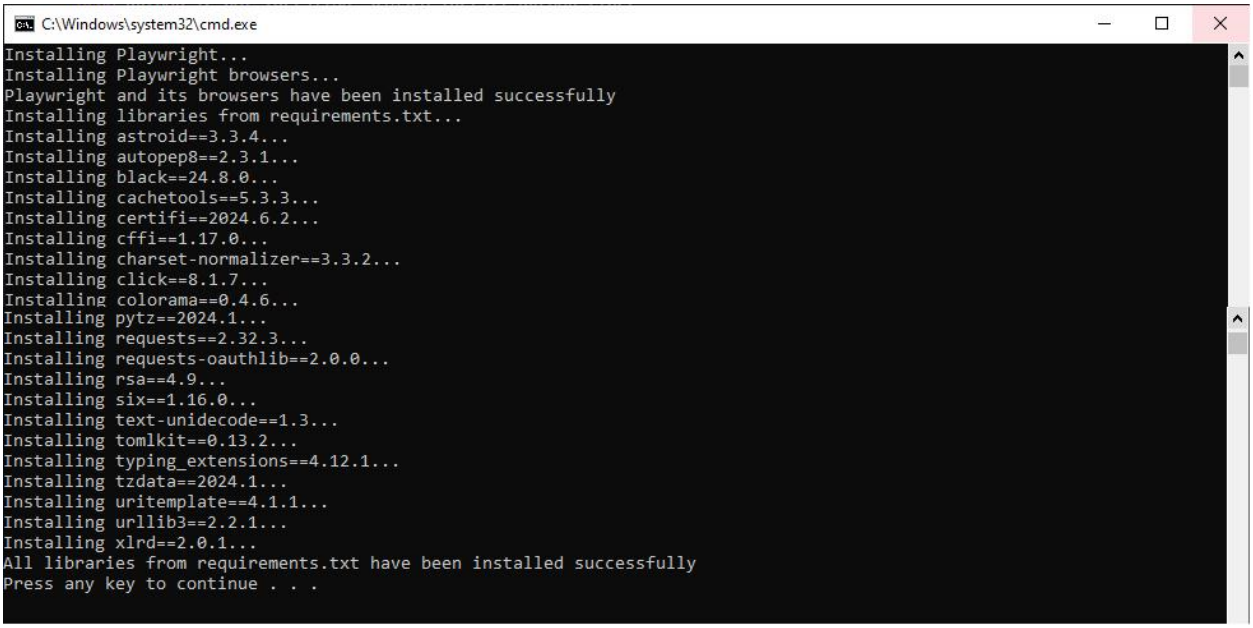


➢ Step3: Install Playwright and Libraries

- Run **Step2_Install_IDE_VS_Code.bat with administrator:**

  In this step, you will install **Playwright**, the **Playwright browsers**, and the necessary libraries required to run automation.

  If you see a message like the image below, the installation of **Playwright** and the required libraries is complete. You can now run automation.



If you encounter an error like the one below when running automation, use the command **playwright install** directly in the Terminal. Wait for it to finish running, and then run the automation again for it to succeed.



❖ Structure

- ***Auto_Python***
  - conftest.py
  - pytest.ini
  - Run_Auto_<Site_Name>_.bat
  - **Actions (folder)**
    - Action.py
    - Action_<Function>.py (Ex: Action_BA_Setting.py)
  - **API (folder)**
    - API_<Function>.py (Ex: API_BA_Setting.py)
  - **config (folder)**
    - client_secret.json
    - config.py
  - **DataSet (folder)**
    - Data_<Function>.py (Ex: Data_Bet_Setting.py)
  - **Interfaces (folder)**
    - Interface_<Function>.py (Ex: Interface_Bet_Setting.py)
  - **TestSuites (folder)**
    - TestSuites_xx_<TestSuiteName>.py
  - **utils (folder)**
    - api_utils.py
    - db_utils.py
    - page_utils.py
    - utils.py

❖ Explaining

Folder	File Name	Parameter	Comment
	conftest.py		The **conftest.py** file in Python Playwright is used to define fixtures and common configurations, allowing them to be shared across test files without requiring imports. It helps organize and optimize test code and enables automatic detection by pytest, reducing code duplication and improving efficiency in initializing browsers or setting up the test environment.
	pytest.ini		The pytest.ini file helps standardize and manage pytest test configurations in a project, ensuring consistent settings and easy integration of specific customizations.
	Run_Auto_<Site_Name>.bat		This file allows us to run multiple TestSuites by adding multiple commands to it and simply executing the file.
Actions	Action.py		This file is a super class, it contains setup and teardown function. When create new **Action_<Function>.py**, should be Import **Action.py** and mark it as super class.
	Action_<Function>.py		This file should be saved action/function to run testcase and verify step. For create new **Action_<Function>.py** file, please refer to this link

API	API_<Function>.py		This file will be stored **require data** to call api. Data should be saved to variables with type is Dictionary.
config	client_secret.json		The client_secret.json file provides OAuth 2.0 credentials (such as client_id and client_secret) for the application to interact with Google APIs, enabling authentication and granting access to Google resources. **Note:** • **Security**: Do not share or store this file in a public repository. • If exposed, revoke and regenerate the credentials in the Google Cloud Console
	config.py	This file will be saved setting for run test.	
		timeout	Default timeout (**second**) for wait element
		headless	Test will be displayed browser on monitor of headless = **False** (Headful) or else.
		env	Determine environment will be run: **local, sit, uat, pro**.
		site	Determine site will be run. Using for send messages to telegram or MS Team
		isUseBrowser	Flag = **True** if you need test on browser.
		isConnectDB	Flag = **True** if you need test on db This flag supports local only.
		browser	The **browser** value specifies the browser Playwright will use (**chromium, firefox, or webkit**) to

			run automation tests.
		isUseAPI	Flag = **True** if you need to call api.
		botToken	Token to connect bot telegram
		chatid	Array list of chatid channel. Using for send message to Telegram
		conversationID	It is the ID of a personal or group conversation in MS Teams for the BOT to send notifications when automation runs are completed.

		database dict	Save account to connect db.
			database =
			{
			“Username” : {
			“local” : “testdemo”
			},
			“Password”: {
			“local” : “testdemo”
			}
			}

		urlSite	Save url link for testing project.
			By default, it should be stored as bellow: <Site> = {
			“local” : {
			“url” : “<local_llink>”
			},
			“sit” : {
			“url” : “<sit_link>”
			},
			“uat” : {
			“url” : “<uat_link>”
			},
			“pro” : {
			“url” : “<pro_link>”
			}
			}

DataSet	Data_<Function>.py		This file will be saved **test data** for run test. Data should be saved to variables with type is Dictionary. Because data on different environments are different, structure of variable should be same example **urlSite** on **config.py** file
Interfaces	Interface_<Function>.py		This file will be saved **elements** of relative pages. Data should be saved to variables with type is Dictionary.
TestSuites	TestSuites_xx_<TestSuiteName>.py		This file should be saved **Test Suites** will be run. For create new **TestSuites_xx_<TestCaseName>.py** file, please refer to this link
utils	utils.py		This file will call all libraries in utils folder. After call setup function on **TestCase.py**, you can use utils lib via self.utils. For more details, please refer to this link
	page_utils.py		This library file contains functions to handle on browser. For more details, please refer to this link.
	db_utils.py		This library file contains functions to connect and query on database. For more details, please refer to this link.
	api_utils.py		This library file contains functions to call api method. For more details, please refer to this link.

❖ Usage

Once build testcases for any function, you will implement these steps on folder below:

Config
API
Data
Interfaces
Actions
TestSuites

♦ config

Provide configuration, url to access website to testing.

For Example, if you want to test on local LRF site, we will provide require data as below:

- On **config.py**:

```
setting = {
    "timeout" : 100,
    "headless" : True,
    "env" : "pro",
    "lang" : "en", # using ch for china language
    "site" : "LAP",
    "isUseBrowser" : True,
    "browser" : "chromium", # chromium,firefox, webkit
    "isConnectDB" : True,
    "isTrace" : True,
    "pathTrace" : "Trace¥¥trace.zip",
    "botToken" : "<Token_Bot_Telegram>",
    "chatid" :  ["-<channelID_will_be_sent1>","-<channelID_will_be_sent2>",...],
    "conversationId" : ["<channelID_will_be_sent1>"]
}
```

- Next, we provide <Site> = **lrf** link on this file:

```
lrf = {
    "local" : {
        "url" : "https://lrf.nexdev.net/LRF3/",
        "folder": "/LRF3",
        "url_access_token": "https://bt.amazzu.com/"
    },
    "sit" : {
        "url" : "http://lrf.playthefun.com/V2/",
        "folder": "/V2",
        "url_access_token": "https://bt.amazzu.com/"
    },
    "uat" : {
        "url" : "https://lrf.starixuat.com/LRF4/Authorization/SignIn?=NotAllow",
        "folder": "/LRF4",
        "url_access_token": "https://lapbt1.starixuat.com"
    },
    "pro" : {
        "url" : "https://internal.amazzu.com/Authorization/SignIn?umtoken=2ZJy-
zk4ygITRRXMj_mgIw2",
        "folder": "",
        "url_access_token": "https://bt.amazzu.com"
    }
}
```

♦ API

Syntax file name : API/API_<Function>.py

With api testing, we prepare header, path, param, file, etc… we store static param

Sample file: API_<Function>.py

```
"<ReportName>" : {
        "Function" : {
            "path" : "/Endpoint",
            "header" : {
                "HeaderName1": "ValueHeader1",
                "HeaderName2": "ValueHeader2",
                "HeaderName3": "ValueHeader3",
            },
            "param" : {
                "pram1" : "value1",
                "pram2" : "value2",
```

```
                ....
                "pramN" : "valueN",
            }
        },
```

Ex: We test api update **Report** function for Winloss Analysis, we will create file API_Reports.py in API folder with content as below.

- **path**: Specifies the API endpoint where requests are sent to retrieve analysis data.
- **header**: Provides necessary information for the API, such as content type, authentication (token), and security (CORS).
- **param**: Passes filtering and pagination parameters (dates, products, currencies, status) to ensure the API returns accurate data as requested.

```
API_Reports = {
    "WinLossAnalysis" : {
        "GetData" : {
            "path" : "/Proxy/Report/api_WinlossAnalysis_GetData",
            "header" : {
                "X-Requested-With" : "XMLHttpRequest",
                "Accept": "application/json, text/plain, */*",
                "__RequestVerificationToken": "",
                "Sec-Fetch-Site": "same-origin",
                "Content-Type": "application/json;charset=utf-8"
            },
            "param" : {
                "PageSize":2000,
                "PageIndex":1,
                "FromDate":"03/12/2024",
                "ToDate":"03/12/2024",
                "ProductCodes":"SB,NG,VS,CS,KN,AB,VB,LT,CC,VRG,SAG,TG,SBC,SXG,PGS,
MXG,VGA,AP,APM",
                "SearchUserName":"",
                "SearchMode":1,
                "CurrencyIdList":"9,97,82,41,13,6,12,15,51,61,32,71,45,70,2,4,3,2
08,20",
                "CurrencyCodes":"All",
                "BaseCurrency":"EUR",
                "StatusIds":"ALL",
```

```
                ..............
            }
        }
    }
}
```

HOW TO USE:

Import the module:
The script uses the API structure from the API_Reports module. First, import the API structure into your working file:

```
from API import API_Reports
```

Create an API dictionary:
Create a dictionary API_Dict containing the required information from API_Reports. For example, retrieve data related to **WinLossAnalysis**:

```
API_Dict = {
    "WinLossAnalysis" : API_Reports.API_Reports["WinLossAnalysis"]
}
```

Extract necessary details:
Extract the key information to send the API request:

- **header:** Contains authentication and data format information.
- **path:** The API endpoint where the request will be sent.
- **param (Data):** Required parameters and data for the API to process.

```
header = API_Dict["WinLossAnalysis"]["GetData"]["header"]
path = API_Dict["WinLossAnalysis"]["GetData"]["path"]
Data = API_Dict["WinLossAnalysis"]["GetData"]["param"]

res = self.utils.api.post(url = url, path = path, header = header, data=Data)
print(str(len(res.json()["data"])))
```

With dynamic params, we should prepare it before calling api function

♦ Data

Syntax file name: Data/Data_<Function>_Page.py

Account, data test, etc… any data we store to use on test case should be stored here.

Sample file: Data_<Function>_Page.py

```
Data = {
    "en": {
        "variableName1": "value1",
        "variableName2": "value2",
        ..........
        "variableNameN": "valueN",
    },
    "ch": {
        "variableName1": "value1",
        "variableName2": "value2",
        ..........
        "variableNameN": "valueN",
}
```

Ex: To test **Reports Win Loss Analysis**, we will create file **Data_Reports_Win_Loss_Analysis.py** in DataSet folder with content as below

```
Data = {
    "en": {
        "Group_BreakMode": {"8,2","1,1","2,2","3,1","4,1","5,1","6,2","7,2",},
        "lblDateRange": "Date",
        "btnSubmit" : "Submit",
        "lblGroupInOriginalCurrency": "Display group breakdown in original
currency",
        "lblExcludeTestData": "Exclude Test Data",
        "lblViewCashOutOnly": "View Cash Out Only",
        "lblUsername": "Username",
        "lblProduct": "Product",
        "lblBettype": "Bet type",
        "lblSport": "Sport",
        "lblPlatform": "Platform",
        "lblStatus": "Status",
        "lblLiveStatus": "Live Status",
        "lblCurrency": "Currency",
        "lblBaseCurrency": "Base Currency",
        "lblVIPPlayerOnly": "VIP Player Only",
        "LiveStatusItems": ["All","Non-Live","Live","Mix Live & Non-Live"],

    },
    "ch": {
        "lblDateRange": "日期",
```

```
        "btnSubmit" : "提交",
        "lblGroupInOriginalCurrency": "以原始货币显示",
        "lblExcludeTestData": "排除测试数据",
        "lblViewCashOutOnly": "只查看实时兑现",
        "lblUsername": "用户名",
        "lblProduct": "产品",
        "lblBettype": "投注类别",
        "lblSport": "运动",
        "lblPlatform": "平台",
        "lblStatus": "状态",
        "lblLiveStatus": "滚球状态",
        "lblCurrency": "货币",
        "lblBaseCurrency": "基础货币",
        "lblVIPPlayerOnly": "只显示 VIP 玩家",
        "LiveStatusItems": ["全部","非滚球(不含混合)","滚球(不含混合)","混合过关"],

    }
}
```

HOW TO USE:

Import the module:
The script uses data structures from the **Data_Reports_Win_Loss_Analysis** module. First, import the data into your working file:

```
from DataSet import Data_Reports_Win_Loss_Analysis
```

Create the Dataset dictionary:
Create a dictionary **DataSet** containing the necessary information for Win Loss analysis. Here, the data is retrieved from **Data_Reports_Win_Loss_Analysis** based on the language specified by the **lang** variable:

```
DataSet = {
    "Win_Loss_Analysis" : Data_Reports_Win_Loss_Analysis.Data[lang],
}
```

Extract the Live Status Items:
Retrieve the LiveStatusItems list from the DataSet:

Verify elements and checkboxes:
Use a **for** loop to iterate through each item in **LiveStatusItems**:

- **Verify element existence:** Call the **verify_element_exist_on_frame** function to check if the element corresponding to the live status exists in the main frame (**frmMain**). XPath is used to locate the element:

```
LiveStatusItems = DataSet["Win_Loss_Analysis"]["LiveStatusItems"]
for LiveStatus in LiveStatusItems:
      self.utils.page.verify_element_exist_on_frame(Interface["Frames"]["frmMain"]
, "//div[@style='display: block;']/ul/li/label[@class='option-label' and
text()='"+LiveStatus+"']")
```

♦ Interfaces

Syntax file name: Interfaces/Interface_<Function>_Page.py.

These elements on page and Regex, jsonpath to detect value should be stored here.

Sample file: Interface_<Function>_Page.py.

```
Data = {
    "en": {
        "variableName1": "XPath1",
        "variableName2": "XPath2",
        ..........
        "variableNameN": "XPathN",
    },
    "ch": {
        "variableName1": "XPath1",
        "variableName2": "XPath2",
        ..........
        "variableNameN": "XPathN",
}
```

Ex: We capture elements on Reports Win Loss Analysis on LAP, we will create file **Interface_Reports_Win_Loss_Analysis.py** in Interfaces folder with content as below:

This file contains definitions of user interface elements (UI elements) for a web application, used for automating user interface testing. These elements are defined for two languages: **English (en)** and **Chinese (ch)**. Each element is identified using **XPath**, which helps to locate the element on the web page.

```
Elements = {
    "en": {

        "DateRange": "//input[@id='DateRange']",
        "lblDateRange":
"//input[@id='DateRange']//parent::div//parent::div//preceding-
sibling::label",
        "btnSubmit" : "//button[@class='btn btn-submit' and text()='Submit']",
        "btnClearAllFilters" : "//button[text()='Clear All Filters']",
        "icoLoading": "//div[@id='container']/div[@style='display:
block;']/div[@class='icon-loading']",
        "icoExpandCollapes": "//div[@id='container']/div[@style='display:
block;']/div[@class='icon-loading']",
        "ddlPageSize": "//select[@class='pagesize-select']",
    },
    "ch": {
        "DateRange": "//input[@id='DateRange']",
        "lblDateRange":
"//input[@id='DateRange']//parent::div//parent::div//preceding-
sibling::label",
        "btnSubmit" : "//button[@class='btn btn-submit' and text()='提交']",
        "btnClearAllFilters" : "//button[text()='清除所有筛选']",
        "icoLoading": "//div[@id='container']/div[@style='display:
block;']/div[@class='icon-loading']",
        "icoExpandCollapes": "//div[@id='container']/div[@style='display:
block;']/div[@class='icon-loading']",
        "ddlPageSize": "//select[@class='pagesize-select']",
}
```

♦ Actions

Syntax file name: Actions/Action_<Function>.py.

- This file will contain the main functions and classes to perform automated tests using the pre-built functions in the utils.py, page_utils.py, api_utils.py and db_utils.py files. You can use the functions in the classes of other Actions files such as Action Common,... And you can import additional libraries needed to write automated test scripts.

Sample file: Action_<Function>.py

```
from Actions import Action
import sys
# Import data,elements. Should add based on your function you define
from config import config
env = config.setting['env']
lang = config.setting['lang']
from Interfaces import Interface_Common
from DataSet import Data_Common
import time
from datetime import datetime, timedelta
import pytz
import utils.utils as utils
import json
from API import API_Reports
from lxml import html
import os
from os.path import dirname as d
from os.path import abspath, join
root_dir = d(d(abspath(__file__)))

# get Data from these import files
Interface = {
    "Main_Menu" : Interface_Common.Element_Menu_LRF[lang],
    "Main_Menu_LAP" : Interface_Common.Element_Menu[lang],
    "Frames" : Interface_Common.Element_Frames_LRF,
    "FramesLAP" : Interface_Common.Element_Frames,
    "Common" : Interface_Common.Elements[lang],
}

DataSet = {
    "Common" : Data_Common.Data_Common[env][lang],
    "CommonText" : Data_Common.DataText[lang],
    "Win_Loss_Analysis" : Data_Reports_Win_Loss_Analysis.Data[lang],
    "TestCase_spread_sheet_id": Data_Common.TestCase_Spread_Sheet_Id,
    "TestCase_Auto_Status": Data_Common.TestCase_Auto_Status
}

class <Function_Name>(Action.Common):
    def setup(self):
        super().setup()

    def referUtils(self, Utils):
        return super().referUtils(Utils)
```

```
    def tearDown(self):
        super().destroy()
        self.utils.update_duration(DataSet["spread_sheet_id"],"Checklist")

    def <Test_Case_Name>(self,testcase):
        subAutoTestCase = '' # save detail sub test case   subAutoTestCase
        mainChecklist = ["<mainChecklist_01>",
                         "<mainChecklist_02>",
                         ….……..
                         "<mainChecklist_N",

                         ] #save checklist
        indexChl = 0 #save latest checklist running
        try:
            {Script to check 'Subtestcase_01 of <mainChecklist_01> here}

            subAutoTestCase = 'Subtestcase_01 of <mainChecklist_01>'

            {Script to check 'Subtestcase_02 of <mainChecklist_01> here}

            subAutoTestCase = 'Subtestcase_02 of <mainChecklist_01>'

            self.utils.update_result_to_ggsheet(DataSet["spread_sheet_id"],
mainChecklist[indexChl], testcase, subAutoTestCase, DataSet["auto_status"][0])
                indexChl = indexChl + 1
                subAutoTestCase = ""

            #### (THIS SCRIPT TO UPDATE STATUS IN GG SHEET)

        except Exception as e:
            # print(indexChl) # = 1
            self.utils.update_result_to_ggsheet(DataSet["TestCase_spread_sheet_id
"], mainChecklist[indexChl], testcase, subAutoTestCase,
DataSet["TestCase_Auto_Status"][1])
            indexChl = indexChl + 1 # = 2
            if indexChl < len(mainChecklist):
                for i in range(indexChl, len(mainChecklist)):
                    if i >= len(mainChecklist):
                        break
                    self.utils.update_result_to_ggsheet(DataSet["TestCase_spread_
sheet_id"], mainChecklist[i], testcase, "", DataSet["TestCase_Auto_Status"][2])
            self.utils.page.extract_TestCaseFailed(e)
```

Ex: We prepare function for Reports Win Loss Analysis, we will create file **Action_Reports_Win_Loss_Analysis.py** in Actions/Report folder and follow the steps below:

Purpose:
This file is primarily used to automate Win Loss Analysis report testing, including UI testing, API testing, and data comparison between different reports.

Steps to Create the Script:

I. Import the necessary libraries and modules:

a) Import functions and classes from other modules (Actions, config, Interfaces, DataSet, API, utils, pytz, json, os, lxml.html, datetime, timedelta, time).

```
# Need Import to use function on TestCase Function
from Actions import Action
import sys
# Import data,elements. Should add based on your function you define
from config import config
from Interfaces import Interface_Common
from Interfaces import Interface_Reports_Win_Loss_Analysis
from Interfaces import Interface_Reports_Licensee_Win_Loss
from Interfaces import Interface_Reports_Live_Report
from Interfaces import Interface_Reports_Cash_Out_Win_Loss
from Interfaces import Interface_Reports_Saba_Virtual_Sports_Win_Loss
from DataSet import Data_Reports_Win_Loss_Analysis
from DataSet import Data_Common
from Actions import Action_Reports_Licensee_Win_Loss
from Actions import Action_Reports_Live_Report
from Actions import Action_Reports_Cash_Out_Win_Loss
from Actions import Action_Reports_Saba_Virtual_Sports_Win_Loss
import time
from datetime import datetime, timedelta
import pytz
import utils.utils as utils
import json
from API import API_Reports
from lxml import html
import os
from os.path import dirname as d
from os.path import abspath, join
root_dir = d(d(abspath(__file__)))
```

b) Set up environment and language settings.

```
env = config.setting['env']
lang = config.setting['lang']
```

II. Define Interface and Data Mappings:

a) Create dictionaries (Interface, DataSet, API_Dict, Regex) to map UI elements, data sets, API endpoints, and regular expressions.

```
# get Data from these import files
Interface = {
    "Main_Menu" : Interface_Common.Element_Menu_LRF[lang],
    "Main_Menu_LAP" : Interface_Common.Element_Menu[lang],
    "Frames" : Interface_Common.Element_Frames_LRF,
    "FramesLAP" : Interface_Common.Element_Frames,
    "Common" : Interface_Common.Elements[lang],
    "Win_Loss_Analysis" : Interface_Reports_Win_Loss_Analysis.Elements[lang],
    "Licensee_Win_Loss" : Interface_Reports_Licensee_Win_Loss.Elements[lang],
    "Live_Report" : Interface_Reports_Live_Report.Elements[lang],
    "Cash_Out_Win_Loss" : Interface_Reports_Cash_Out_Win_Loss.Elements[lang],
    "Saba_Virtual_Sports_Win_Loss" :
Interface_Reports_Saba_Virtual_Sports_Win_Loss.Elements[lang]
}

DataSet = {
    "Common" : Data_Common.Data_Common[env][lang],
    "CommonText" : Data_Common.DataText[lang],
    "Win_Loss_Analysis" : Data_Reports_Win_Loss_Analysis.Data[lang],
    "TestCase_spread_sheet_id": Data_Common.TestCase_Spread_Sheet_Id,
    "TestCase_Auto_Status": Data_Common.TestCase_Auto_Status
}

API_Dict = {
    "WinLossAnalysis" : API_Reports.API_Reports["WinLossAnalysis"]
}

Regex = {
    "WinLossAnalysis" : Interface_Common.Regex
}
```

III. Create the Win_Loss_Analysis Class:

a) Inherit from a common action class (Action.Common).

```
class Win_Loss_Analysis(Action.Common):
```

b) Define methods for setup, teardown, and utility references.

```
def setup(self):
        super().setup()
        Licensee_Win_LossFunc =
Action_Reports_Licensee_Win_Loss.Licensee_Win_Loss()

        self.Licensee_Win_LossFunc = Licensee_Win_LossFunc

    def referUtils(self, Utils):
        return super().referUtils(Utils)

    def tearDown(self):
        super().destroy()
```

IV. Implement Verify_Win_Loss_Analysis Method:

a) Navigate to the login page.

```
Action.Common.loginLRF(self)
```

b) **Action.Common.LoginLRF** use **loginLRF** in Common Action

```
def loginLRF(self):
        DataSet["login_page_lrf"]["Password"] =
utilities.decrypt(DataSet["login_page_lrf"]["encrypted"],
DataSet["login_page_lrf"]["Password"], DataSet["login_page_lrf"]["salt"])
        self.utils.page.navigate(config.lrf[env]["url"])
        self.utils.page.input("//input[@id='userName']",
DataSet["login_page_lrf"]["Username"])
        self.utils.page.input("//input[@id='passWord']",
DataSet["login_page_lrf"]["Password"])
        self.utils.page.input("//input[@id='captcha']", "1710")
        self.utils.page.click("//input[@value='Login']")
```

Some info in **Action_Reports_Win_Loss_Analysis:**

```
def Verify_Win_Loss_Analysis(self):
    try:
        Action.Common.loginLRF(self)
        Action.Common.selectLicensee(self, Licensee)
        time.sleep(5)
        Action.Common.gotoMenu(self, "", Interface["Main_Menu"]["Reports"]["lnkWinLossAnalysis"])
        time.sleep(20)

        ####Verify text of element's label
        self.Verify_Text_Of_Labels()

        ####Verify default value of all elements
        self.Verify_Default_Value_Of_Elements()

        #radio Group by Agent exists & uncheck
        self.utils.page.verify_element_exist_on_frame(Interface["Frames"]["frmMain"], Interface["Win_Loss_Analysis"]["rdoGroupByAgent"])
        self.utils.page.verify_checkbox_on_frame(Interface["Frames"]["frmMain"], Interface["Win_Loss_Analysis"]["rdoGroupByAgent"],False)
        ##################

        ####Verify select/unselect All Products then Sports / Bettype dropdown list are changed as well
        #If Uncheck All Product then Sports/Bettype dropdown list unchecked all
        self.Verify_Status_Of_All_Items_In_Dropdown_List(Interface["Win_Loss_Analysis"]["ddlProduct"],False)

        #Verify CashOut dropdown is disabled when uncheck Sportsbook product
        self.utils.page.verify_element_exist_on_frame(Interface["Frames"]["frmMain"], "//div[@aria-activedescendant='react-select-CashOutMode--value' and @aria-dis

        #Verify Sport / Bettype dropdown list show N/A
        self.utils.page.verify_element_contain_text(Interface["Frames"]["frmMain"], Interface["Win_Loss_Analysis"]["lblSport_NoItemSelected"],DataSet["CommonText"]
        self.utils.page.verify_element_contain_text(Interface["Frames"]["frmMain"], Interface["Win_Loss_Analysis"]["lblBettype_NoItemSelected"],DataSet["CommonText

        #Verify Submit button is disabled when uncheck all products
        self.utils.page.verify_element_enabled_on_frame(Interface["Frames"]["frmMain"],Interface["Win_Loss_Analysis"]["btnSubmit"],False)
```

♦ TestSuites

Syntax file name: TestSuites/TestSuites_xx_<Function>.py

This file is an automated test script that uses the **unittest** library to test the functionality of the Function or Report. It sets up the test environment, executes the test, and cleans up after the test is complete. The test methods defined in this class will use actions from the **Action_<Function>** module.

Sample file: TestSuites_xx_<Function>.py

```
# Must import, please don't fix it
import sys
import os
from os.path import dirname as d
from os.path import abspath, join
root_dir = d(d(abspath(__file__)))
sys.path.append(root_dir)
import unittest
unittest.TestLoader.testMethodPrefix = "TestCase"
unittest.TestLoader.sortTestMethodsUsing = None

from config import config
```

```
# Should Add based on your function you define
from Actions import Action_<Action_<Function>.py>
from Actions import Action
import utils.page_utils as page

from Interfaces import Interface_Common
from Interfaces import Interface_<Function>.py

from DataSet import Data_Common
import time

class <Function_Name>(unittest.TestCase):
    @classmethod
    def setUpClass(self):
        <FunctionName>Func = Action_<Function_Name>.<ClassOfFuntion>()
        <FunctionName>Func.setup()
        self.<FunctionName>Func = <FunctionName>Func

        filename = os.path.abspath(__file__).split("\\")
        TraceFile = "Trace\\" + filename[len(filename)-1].replace(".py",".zip")
        config.setting['pathTrace'] = TraceFile
        print("Filename: " + TraceFile)

    @classmethod
    def tearDownClass(self):
        self.<FunctionName>Func.tearDown()
    def TestCase_001_TESTCASE_NAME(self):
        self.<FunctionName>Func.Method_In_<ClassOfFuntion>()
```

Ex: We prepare TestSuites for **Reports Win Loss Analysis**, we will create file **TestSuites_03_LRF_Reports_Win_Loss_Analysis** in TestSuites folder and follow the steps below:

Steps to Create the Script:

I. Import root path (require)

```
import sys
from os.path import dirname as d
from os.path import abspath, join
root_dir = d(d(abspath(__file__)))
```

```
sys.path.append(root_dir)
```

II.Import unittest (require)

```
import unittest
unittest.TestLoader.testMethodPrefix = "TestCase"
unittest.TestLoader.sortTestMethodsUsing = None
```

- Explaining:

  - unittest.TestLoader.testMethodPrefix = “TestCase” : Define these test cases will be run if FunctionName contains TestCase. Ex: When you create test case Verify Bet Limit Retriction For Sportbooks:

```
def TestCase_001_Verify_Win_Loss_Analysis(self):
        self.Win_Loss_AnalysisFunc.Verify_Win_Loss_Analysis()
```

  - unittest.TestLoader.sortTestMethodsUsing = None : No using sort test method, Test case will be run up to down.

III. Import configuration and Action modules

```
from config import config
env = config.setting['env']
lang = config.setting['lang']

# Should Add based on your function you define
from Actions import Action_Reports_Win_Loss_Analysis
from Actions import Action
import utils.page_utils as page

from Interfaces import Interface_Common
from Interfaces import Interface_Reports_Win_Loss_Analysis

from DataSet import Data_Common
import time
```

IV. Create class for this Test case

```
class <Function>(unittest.TestCase):

        @classmethod
```

```
    def setUpClass(self):
        <Function>Func=Action_<Function>.<Function>()
        <Function>Func.setup()
        self.<Function>Func = <Function>Func
        self.utils = <Function>Func.utils
    @classmethod
    def tearDownClass(self):
        self.<Function>Func.tearDown()
```

- Explaining:
  - setUpClass is a Setup function.
    - It will create and initialize Function class then assign to self.<Function>Func.
    - In addition to self.<Function>Func, utilities class is created and initialized then assign to self.utils
  - tearDownClass is a tearDown function, it will clear cache to the system is ready to run another test suite

With <Function> = Win_Loss_Analysis, we have:

```
class Win_Loss_Analysis(unittest.TestCase):
    @classmethod
    def setUpClass(self):
        Win_Loss_AnalysisFunc =
Action_Reports_Win_Loss_Analysis.Win_Loss_Analysis()
        Win_Loss_AnalysisFunc.setup()

        self.Win_Loss_AnalysisFunc = Win_Loss_AnalysisFunc
```

```
        filename = os.path.abspath(__file__).split("\\")
        TraceFile = "Trace\\" + filename[len(filename)-1].replace(".py",".zip")
        config.setting['pathTrace'] = TraceFile
        print("Filename: " + TraceFile)

    @classmethod
    def tearDownClass(self):
        self.Win_Loss_AnalysisFunc.tearDown()
```

V. Add functions to run test case.

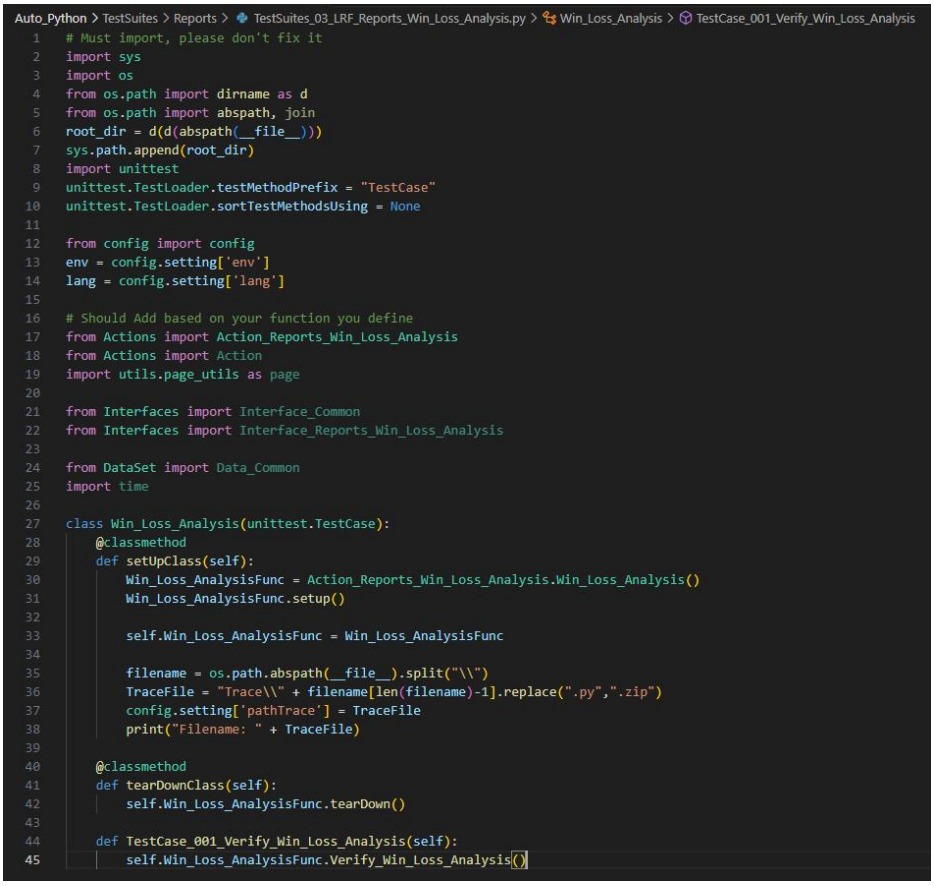
- We should add below this class

  Example with **Verify_Win_Loss_Analysis** testcase:

```
def TestCase_001_Verify_Win_Loss_Analysis(self):
        self.Win_Loss_AnalysisFunc.Verify_Win_Loss_Analysis()
```

Note: For **utils** function, please refer to this link

All info of TestSuites_03_LRF_Reports_Win_Loss_Analysis.py:



❖ Dictionary

Utilities class contains common functions to activate with web driver, db, api, etc... and more. It should be updated if I receive any requests to write scripts easily.

- **utils.py**
- **page_utils.py**
- **api_utils.py**
- **db_utils.py**

utils:

File Name	Function	Description	Example
utils.py	This class will create and initialize api, db, page class to use		
	clear_browser	To clear cookies on browser	self.utils.clear_browser()
	destroy	To close browser	self.utils.destroy()
	should_be_equal	To verify <actual> should be equal <expected>. The System will set Failed Test case if <actual> isn't equal <expected>	self.utils.should_be_equal(<actual> <expected >)
	should_be_not_equal	To verify <actual> should be not equal <expected>. The System will set Failed Test case if <actual> is equal <expected>	self.utils.should_be_not_equal(<actual>,<expected>)
	extract_FunctionName prepare_message send_message_to_telegram Send_message_to_team	This function supports sending telegram messages (this framework will send **telegram** messages after checking based on **botToken** and **chatid** in config.py or send to **MS Team** by **conversationId**), we don't care about it	

page_utils.py

File Name	Function	Description	Example
page_utils.py	This variable has all functions of page_utils.py		
	navigate	Go to <url>	self.utils.page.navigate("<url>")
	get_current_url	Return current url on browser	url = self.utils.page.get_current_url()
	reload	Reload this page	self.utils.page.reload()
	sleep	Wait a few milliseconds	self.utils.page.sleep(5000) # wait 5 seconds
	input	Paste <text> into <element> This function works with <input>, <textarea>, <contenteditable> element only	self.utils.page.input("<element>" ,"<text>")
	input_on_frame	Look like input function but control on selected frame	self.utils.page.input_on_frame("< frame_name> ","<element>","<t ext>")
	select_options	Select <item> for an <element> <item> can be index, value, label name and can select multiple attribute	self.utils.page.select_options("<el ement>","<te xt>")
	select_options_on_fra me	Look like select_options function but control on selected frame	self.utils.page.select_options_on_ frame("<fram e_name>","<element>","<text>")
	get_attribute	Returns element attribute value	val = self.utils.page.get_attribute("<ele ment>",<attrib ute_name>)
	get_attribute_on_fram e	Look like get_attribute function but control on selected frame	val = self.utils.page.get_attribute_on_fr ame("<frame _name>","<element>",<attribute

			_name>)
	get_text	Return text value of element	text = self.utils.page.get_text(“<element>”)
	get_text_on_frame	Look like get_text function but control on selected frame	text = self.utils.page.get_text_on_frame(“<frame_na me>”,“<element>”)
	get_textHTML	Return the HTML content (inner HTML) of an element	text = self.utils.page.get_textHTML(“<element>”)
	get_textHTML_on_frame	Look like get_textHTML function but control on selected frame	text = self.utils.page.get_textHTML_on_frame(“<frame_name>”,“<element>”)
	get_value_on_frame	Return value of input or textarea or select element	val = self.utilspage.get_value_on_frame(“<frame_name>”,“<element>”)
	match_by_regex	Return True if regex pattern match on HTML content, else Return False	bl = self.utils.page.match_by_regex(“<regex_patter n>”)
	match_by_regex_on_frame	Look like match_by_regex function but control on selected frame	bl = self.utils.page.match_by_regex_on_frame(“<frame_name>”,“<regex_pattern>”)
	extract_by_regex	Return text match with regex_pattern, if using group, you need provide <group> to get correct data	1. str = self.utils.page.extract_by_regex(“<regex_patter n>”) 2. strGroup = self.utils.page.extract_by_regex(“<regex_patter n>”,<group>)
	extract_by_regex_on_frame	Look like extract_by_regex function but control on selected frame	1. str = self.utils.page.extract_by_regex_on_frame(“<frame_name>”,“<regex_pattern>”) 2. strGroup = self.utils.page.extract_by_regex_on_frame(“<fr

			ame_name>”,“<regex_pattern>”, <group>)
	press_key	Press key from keyboard to element By default, key will press on body HTML page	1. self.utils.page.press_key(“<Key>”) 2. self.utils.page.press_key(“<Key>”,“<element>”)
	press_key_on_frame	Look like press_key function but control on selected frame	1. self.utils.page.press_key_on_frame(“<frame_n ame>”,“<Key>”) 2. self.utils.page.press_key_on_frame(“<frame_name>”,“<Key>”,“<element>”)
	wait_element	Wait element appears, default wait 30 seconds The System will set Failed Test case if timeout	self.utils.page.wait_element(“<element>”)
	wait_element_on_frame	Look like wait_element function but control on selected frame	self.utils.page.wait_element_on_frame(“<frame_name>”,“<element>”)
	wait_load_page	By default, This function will wait to the page DOM has been loaded	self.utils.page.wait_load_page()
	wait_frame_load	Look like wait_load_page function but control on selected frame	self.utils.page.wait_frame_load(“<frame_name >”)
	screenshot	Screenshot browser By default, this function will be save image to Image folder with current timestamp is a name file	1. self.utils.page.screenshot() 2. self.utils.page.screenshot(“”)

api_utils.py

File Name	Function	Description	Example
api_utils.py		This variable has all functions of api_utils.py.py	
	get	This function to send a GET request to the url Return a response object Optional parameter include: - path - header - data - param	res = self.utils.api.get("<url>")
	post	This function to send a POST request to the url Return a response object Optional parameter include: - path - header - data (json file) - params (parameter) - formData (application/x-www-form-urlencoded) - multipart (multipart/form-data)	dataJson = " { "<key>" : "<value>" }" res = self.utils.api.post("<url>",data = dataJson)
	extract_text	Return response as text Response get from get or post function	res = self.utils.api.get("<url>") text = self.utils.api.extract_text(res)
	extract_status_code	Return status code of response Response get from get or post function	res = self.utils.api.get("<url>") statuscode = self.utils.api.extract_status_code(res)
	extract_response_json	Return response as json object Response get from get or post function	res = self.utils.api.get("<url>") jsonObject = self.utils.api.extract_response_json(res)
	match_by_reg	Return True if regex pattern	res = self.utils.api.get("<url>")

	ex	matches on text response, else Return False	bl = self.utils.api.match_by_regex(res ,“<regex_patt ern>”)
	extract_by_jsonpath	Return text when jsonpath matches on text response Response get from get or post function Data will be get based on<Index>. By default, <Index> = 0	1. res = self.utils.api.get(“<url>”) text = self.utils.api.extract_by_jsonpath (res,“<json_pa th>”) 2. res = self.utils.api.get(“<url>”) text = self.utils.api.extract_by_jsonpath (res,“<json_pa th>”,<index>)
	match_by_jsonpath	Return True if jsonpath matches on text response, else Return False	res = self.utils.api.get(“<url>”) bl = self.utils.api.match_by_regex(res ,“<json_path> ”)

db_utils.py

File Name	Function	Description	Example
db_utils.py	This variable has all functions of db_utils.py This library run local only		
	connect	Connect to DB	self.utils.db.connect(‘DB215’,‘bodb02’)
	close	Close DB Connection	self.utils.db.close()
	query_and_get_all	Query and get data result Data result is a Dict Array	arDict = self.utils.db.get_all(“<query>”)
	get_value_in_column	Query and get data result via <column_name> Data result is a Array String	arr = self.utils.db.get_value_in_column(“<query>”,“ <column_name>”)
	insert_data_to_table	Insert data to database If you want pass null data on column: - params = { “column_name” :	params = { “column_name_1” : “value_of_column_name_1”,

		None } - params = { “column_name” : “null” } If you want pass date on column - Today : params = { “column_name_date” : “today” } or { “column_name_date” : “Today” } - Yesterday : params = { “column_name_date” : “today - 1” } - The day before n day : params = { “column_name_date” : “today - n” } - Tomorrow : params = { “column_name_date” : “today + 1” } - The day after n day : params = { “column_name_date” : “today + n” } - First day of current month : params = { “column_name_date” : “first_day_of_month” } - Last day of current month : params = { “column_name_date” : “last_day_of_month” } - Date = 2021-11-24 : params = { “column_name_date” : “2021-11-24” }	“column_name_2” : “value_of_column_name_2”, … “column_name_n” : “value_of_column_name_n” } self.utils.db.insert_data_to_table(“<table_nam e>”,params)
	insert_data_from_file	Insert data to database via .sql file Support upto 10 times for each run file This function will replace text row[i][0] = row[i][1] if sql file contains row[i][0]	Ex: list_of_data = [[“TypeOfData”, “Ticket_Cashout”], [“<Username>”, “12BETUUS01002”], [“<SportName>”,“Soccer”], [“<StatusId>”,“112”], [“<EventStatus>”,“completed”]]

			self.utils.db.insert_data_from_file(“<path_to_sql_file>”,<number_of_time>,list_of_data)